Product Optimization & Revenue Contribution Analysis for Afficionado Coffee Roasters

KPI-Based Time-Series Dashboard & Operational Analysis (Portfolio Project)

Project Overview

This project analyses sales records for Afficionado Coffee Roasters to optimise revenue generation. The primary objective is to identify top performing products and pinpoint the least profitable items sold, to quantify revenue contribution by product and category, and to measure revenue concentration across the menu.

Secondary objectives: to support menu simplification and optimization, to identify high impact products, and to highlight low performing products for review or redesign.

The full **Python** implementation used for data cleaning, KPI engineering, and visualization is provided in Appendix A to ensure reproducibility.

Streamlit Code

```
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
import seaborn as sns

st.set_page_config(page_title="Coffee Retail Analytics Dashboard", layout="wide")

# Load dataset
try:
    df = pd.read_csv("cleaned_afficionado_dataset.csv")
    st.success("Dataset loaded successfully!")
except FileNotFoundError:
    st.error("ERROR: cleaned_afficionado_dataset.csv not found in the project folder.")
    st.stop()

st.title("Afficionado Coffee Roasters - Product Revenue Dashboard")

# Sidebar Filters
st.sidebar.header("Filter Options")

category_filter = st.sidebar.multiselect(
    "Select Product Category",
    options=df["product_category"].unique(),
    default=df["product_category"].unique()
)

type_filter = st.sidebar.multiselect(
    "Select Product Type",
    options=df["product_type"].unique(),
    default=df["product_type"].unique()
)

# Apply filters
filtered_df = df[
    (df["product_category"].isin(category_filter)) &
    (df["product_type"].isin(type_filter))
]

# KPI Section
st.subheader("Key Performance Indicators")

total_revenue = filtered_df["revenue"].sum()
total_transactions = filtered_df["transaction_qty"].sum()
avg_revenue = filtered_df["revenue"].mean()

col1, col2, col3 = st.columns(3)
col1.metric("Total Revenue", f"{total_revenue:,.2f}")
col2.metric("Total Units Sold", f"{total_transactions:,}")
col3.metric("Average Revenue per Transaction", f"{avg_revenue:,.2f}")
```

```
# Top Products by Revenue
st.subheader("Top 10 Products by Revenue")
top_products = (
    filtered_df.groupby("product_detail")["revenue"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

fig1, ax1 = plt.subplots(figsize=(10, 6))
sns.barplot(data=top_products, x="revenue", y="product_detail", ax=ax1)
ax1.set_title("Top 10 Revenue-Generating Products")
st.pyplot(fig1)

# Revenue by Category
st.subheader("Revenue by Product Category")
category_revenue = (
    filtered_df.groupby("product_category")["revenue"]
    .sum()
    .sort_values(ascending=False)
    .reset_index()
)

fig2, ax2 = plt.subplots(figsize=(8, 5))
sns.barplot(data=category_revenue, x="revenue", y="product_category", ax=ax2)
ax2.set_title("Revenue Distribution by Category")
st.pyplot(fig2)

# Data Preview
st.subheader("Filtered Dataset Preview")
st.dataframe(filtered_df.head(20))
```

Data validation and Preparation

Analytical reliability and consistency was ensured by performing data validation on the transaction-level dataset "Afficionado Coffee Roasters-Transactions.csv". First the dataset was loaded into a pandas DataFrame. Chronological consistency for a potential time-based analysis was secured by converting the *transaction_time* column into datetime format.

Key variables including *product_id, unit_price,* and *transaction_qty* were systematically validated. The *product_id* column was checked for missing, non-positive, and invalid values, confirming that all identifiers were valid and correctly formatted.

The *unit_price* column, originally stored as an object data type, was converted to a numeric (float64) format. Comma-separated decimal values were standardized by replacing commas with periods, and any non-numeric values were coerced and handled. Since no missing values were present after conversion, no imputation was required. All unit prices were confirmed to be positive.

Transaction quantities (*transaction_qty*) were verified to be positive and within a realistic operational range. Descriptive statistics showed a minimum of 1, maximum of 8, and an average of 1.44 units per transaction, indicating realistic retail purchasing behavior.

The dataset contained 149,116 rows and 11 columns, with no duplicate records detected. Overall, the data was clean, consistent, and suitable for revenue and product performance analysis.

Revenue Calculation and Aggregation

Transaction-level revenue was calculated using the formula:

$$\text{Revenue} = transaction_qty \times unit_price$$

A new revenue column was created to quantify financial contribution per transaction. The revenue was then aggregated across multiple product hierarchies, including:

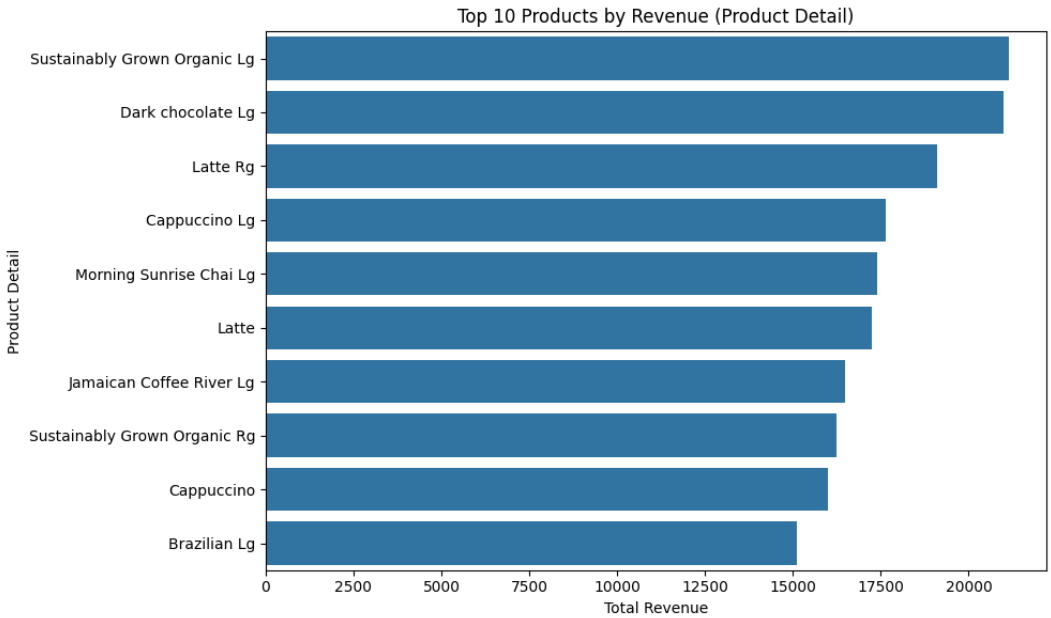
- Product detail
- Product type
- Product category

This multi-level aggregation enabled a comprehensive evaluation of both product popularity and profitability.

Key Findings

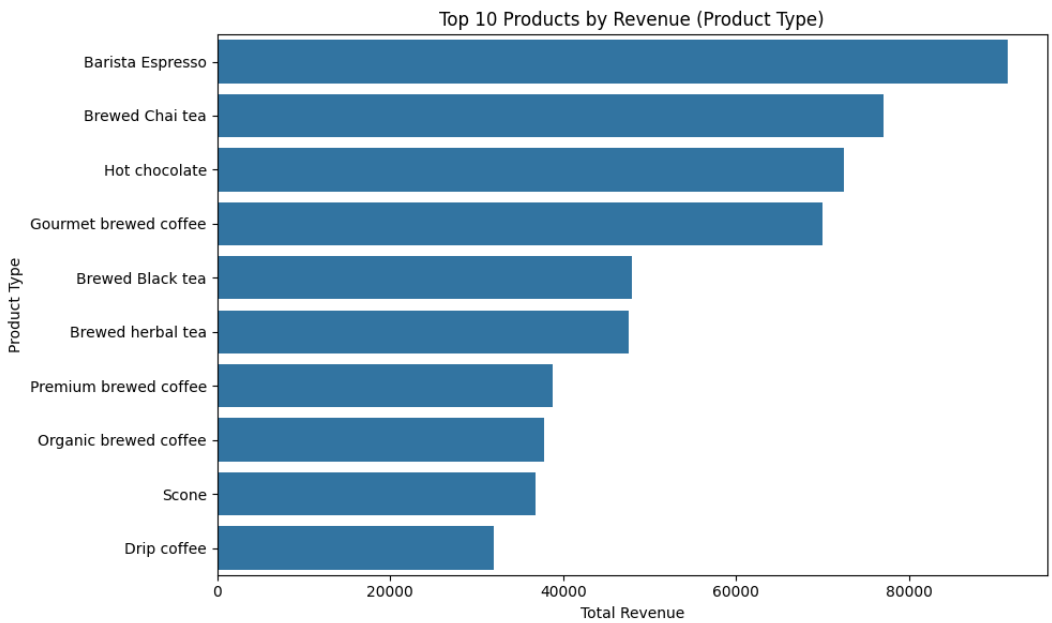
The analysis revealed a clear revenue concentration across a limited number of high-performing products. At the product detail level, items such as “Sustainably Grown Organic Lg” and “Dark Chocolate Lg” emerged as the primary revenue drivers.

Figure 1: Top 10 Products by Revenue



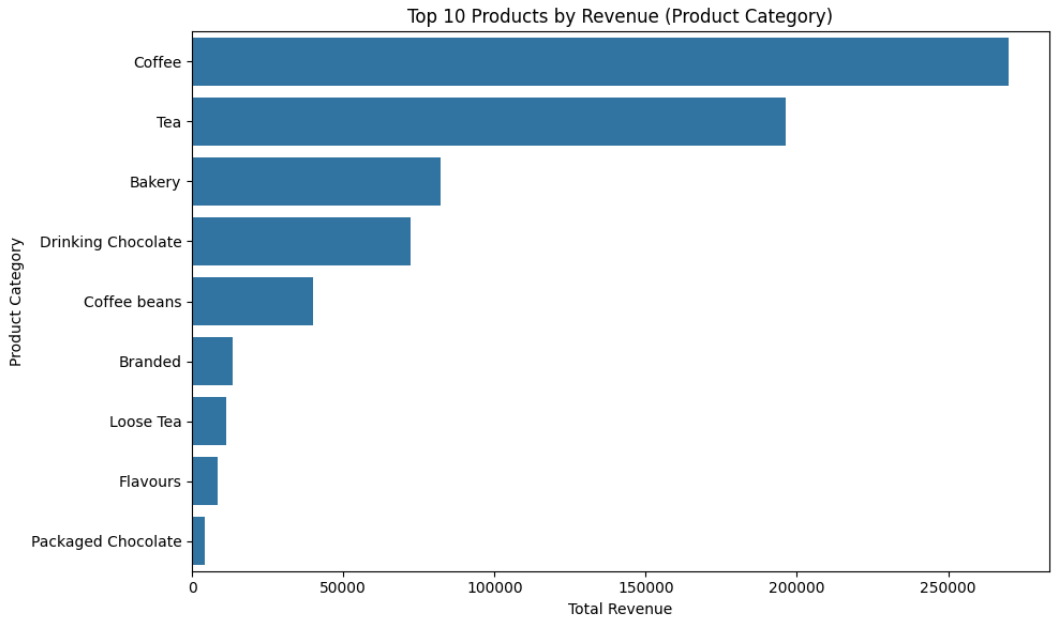
At the product type level, “Barista Espresso”, “Brewed Chai Tea”, and “Hot Chocolate” generated the highest revenue, indicating strong customer preference for core beverage offerings.

Figure 2: Top 10 Products by Revenue by Type



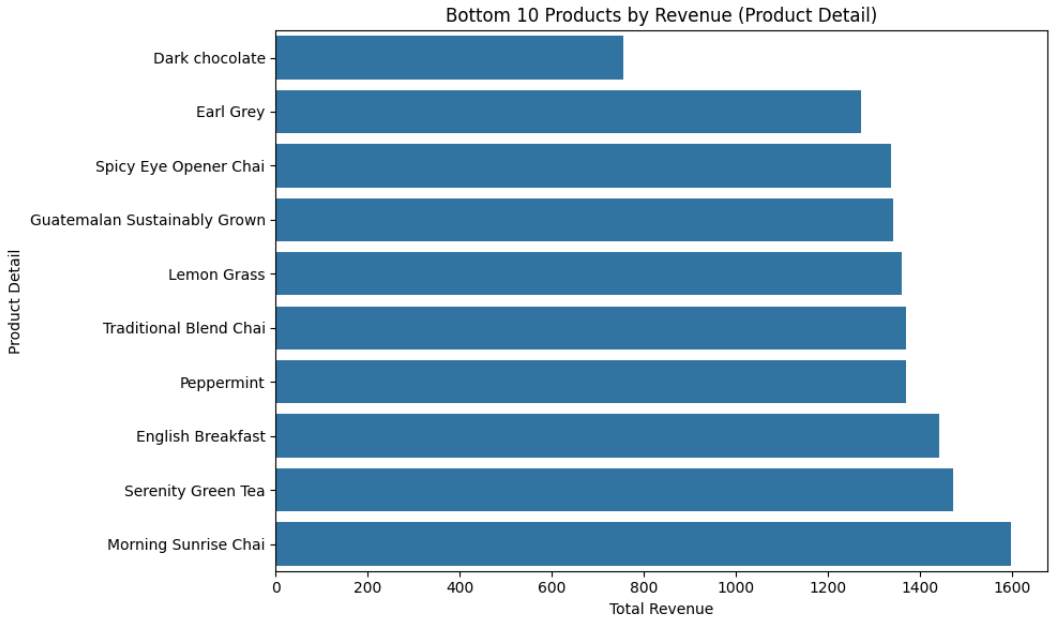
Category-level analysis showed that Coffee, Tea, and Bakery categories contributed the majority of total revenue, highlighting a strong dependency on beverage-led sales in the retail model. In contrast, categories such as Packaged Chocolate, Flavours, and Loose Tea demonstrated significantly lower revenue contribution.

Figure 3: Top 10 Products by Revenue by Category



The bottom-performing products, including items such as Dark Chocolate (small variants) and Earl Grey, exhibited relatively low revenue, suggesting limited demand or lower purchase frequency.

Figure 4: Bottom 10 Products by Revenue



The scatter plot reveals clusters of high-volume low-price items and lower-volume premium products. This indicates that both pricing strategy and sales volume contribute to revenue generation, suggesting opportunities for targeted pricing or promotion strategies.

Figure 5: Revenue vs Unit Sold

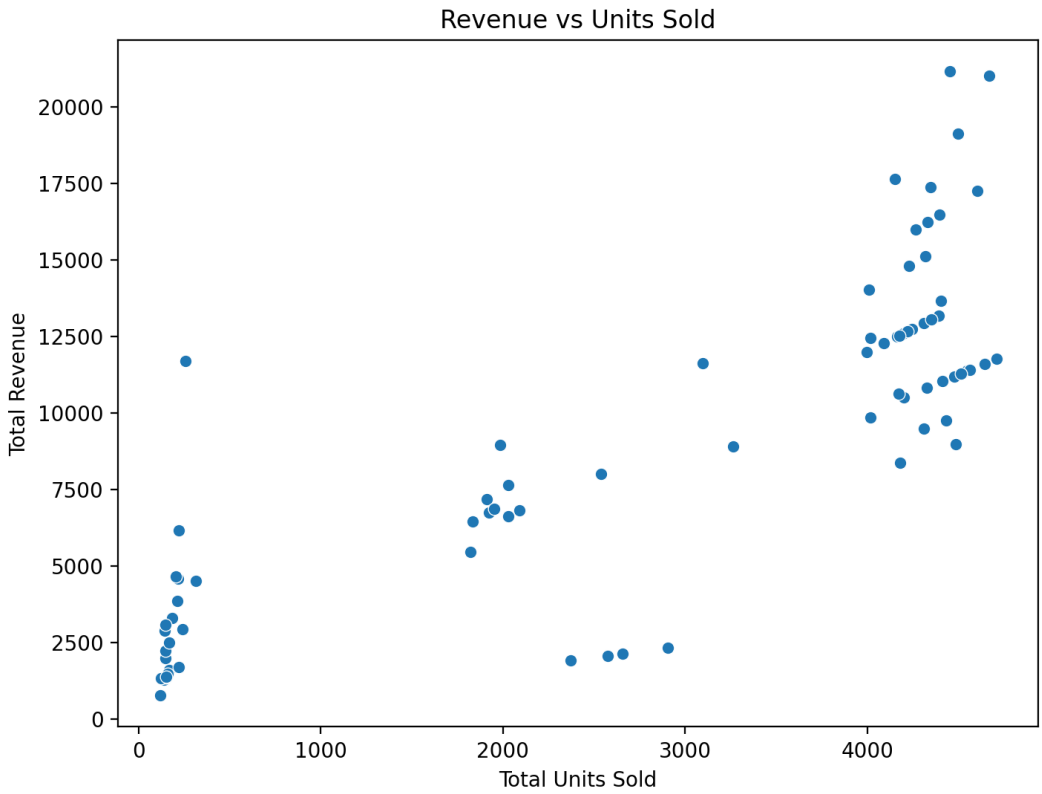
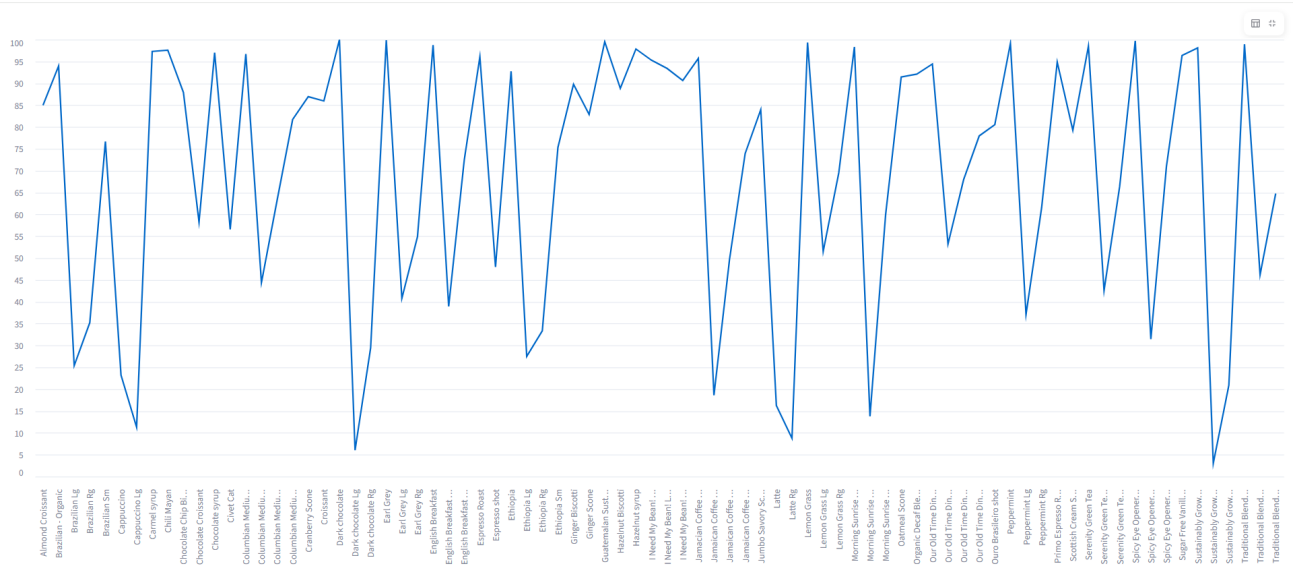


Figure 6: Pareto analysis for Revenue Concentration



Visualization and Business Insights

Bar charts and a scatter plot were generated to visualize the top and bottom 10 performers by product detail, type, and category. These visualizations clearly illustrate revenue concentration and long-tail product behavior within the menu structure.

The results suggest that a limited number of high-impact products act as revenue anchors, while several low-performing items contribute marginally to overall sales. This pattern aligns with the Pareto principle commonly observed in retail analytics.

From a business perspective, the findings support:

- Menu optimization strategies
- Inventory prioritization
- Targeted marketing for high-performing products
- Review or redesign of low-impact items

Conclusion

The project successfully transformed raw transaction-level data into actionable retail insights by focusing on product-level revenue performance. Through data validation, revenue calculation, and hierarchical aggregation, the analysis identified key revenue-driving products and underperforming menu items. An interactive dashboard was also developed using Streamlit to enable dynamic filtering and product-level drill-down analysis.

The findings indicate that a small number of core beverage products act as revenue anchors, while several long-tail products contribute minimally to overall revenue. These insights can support data-driven decision-making in menu optimization, inventory planning, and targeted marketing strategies.

Future improvements could include time-based sales analysis, store location comparisons, customer segmentation, and Pareto (80/20) revenue concentration analysis to further enhance business intelligence capabilities.

Appendix

Appendix A – Python Implementation

```
#pip install pandas scikit-learn matplotlib seaborn

import pandas as pd

file_path = "Transactions.csv"
data = pd.read_csv("Transactions.csv")

data['transaction_time'] = pd.to_datetime(data['transaction_time'], format='%H:%M:%S',
errors='coerce')
print(data.head())

data = data.sort_values(by='transaction_time').reset_index(drop=True)

print("Initial Shape:", data.shape)
print("Columns:", data.columns.tolist())

duplicate_rows = data[data.duplicated()]
print(f"Number of duplicate rows: {duplicate_rows.shape[0]}")
if not duplicate_rows.empty:
  print(duplicate_rows.head())
else:
  print("No duplicate rows found.")

print(f"Data type of 'product_id': {data['product_id'].dtype}")
print(f"Missing values in 'product_id': {data['product_id'].isnull().sum()}")

non_positive_product_ids = data[data['product_id'] <= 0]
if not non_positive_product_ids.empty:
  print("Non-positive 'product_id' values found:")
  print(non_positive_product_ids)
else:
  print("All 'product_id' values are positive.")

print(f"Original data type of 'unit_price': {data['unit_price'].dtype}")
print(f"Missing values in 'unit_price' before conversion: {data['unit_price'].isnull().sum()}")

data['unit_price'] = data['unit_price'].astype(str).str.replace(',', '.', regex=False)
data['unit_price'] = pd.to_numeric(data['unit_price'], errors='coerce')

print(f"\nData type of 'unit_price' after conversion: {data['unit_price'].dtype}")
print(f"NaN values in 'unit_price' after conversion (introduced due to errors):
{data['unit_price'].isnull().sum()}")

if data['unit_price'].isnull().sum() > 0:
  median_unit_price = data['unit_price'].median()
  data['unit_price'].fillna(median_unit_price, inplace=True)
  print(f"Filled NaN values in 'unit_price' with median: {median_unit_price}")
else:
```

```
  print("No NaN values to fill in 'unit_price'.")

non_positive_unit_prices = data[data['unit_price'] <= 0]
if not non_positive_unit_prices.empty:
  print("Non-positive 'unit_price' values found:")
  print(non_positive_unit_prices)
else:
  print("All 'unit_price' values are positive.")


print(f"Data type of 'transaction_qty': {data['transaction_qty'].dtype}")
print(f"Missing values in 'transaction_qty': {data['transaction_qty'].isnull().sum()}")

non_positive_transaction_qty = data[data['transaction_qty'] <= 0]
if not non_positive_transaction_qty.empty:
  print("\nNon-positive 'transaction_qty' values found:")
  print(non_positive_transaction_qty)
else:
  print("\nAll 'transaction_qty' values are positive.")

min_qty = data['transaction_qty'].min()
max_qty = data['transaction_qty'].max()
avg_qty = data['transaction_qty'].mean()

print(f"\nMinimum 'transaction_qty': {min_qty}")
print(f"Maximum 'transaction_qty': {max_qty}")
print(f"Average 'transaction_qty': {avg_qty:.2f}")

data['revenue'] = data['transaction_qty'] * data['unit_price']
print(data.head())

revenue_by_product_detail = (
    data.groupby('product_detail')['revenue']
    .sum()
    .reset_index()
    .sort_values(by='revenue', ascending=False)
)
revenue_by_product_detail_b=revenue_by_product_detail.sort_values(by='revenue',
ascending=True)

print("Top 10 Products by Revenue (product_detail):")
print(revenue_by_product_detail.head(10))
print("Bottom 10 Products by Revenue (product_detail):")
print(revenue_by_product_detail_b.head(10))

revenue_by_product_type = (
    data.groupby('product_type')['revenue']
    .sum()
    .reset_index()
    .sort_values(by='revenue', ascending=False)
)
revenue_by_product_type_b=revenue_by_product_type.sort_values(by='revenue', ascending=True)
```

```
print("Top 10 Products by Revenue (product_type):")
print(revenue_by_product_type.head(10))
print("Bottom 10 Products by Revenue (product_type):")
print(revenue_by_product_type_b.head(10))

revenue_by_product_category = (
    data.groupby('product_category')['revenue']
    .sum()
    .reset_index()
    .sort_values(by='revenue', ascending=False)
)
revenue_by_product_category_b=revenue_by_product_category.sort_values(by='revenue',
ascending=True)

print("Top 10 Products by Revenue (product_category):")
print(revenue_by_product_category.head(10))
print("Bottom 10 Products by Revenue (product_category):")
print(revenue_by_product_category_b.head(10))

print("All Products by Revenue (product_type):")
print(revenue_by_product_type)

print("All Products by Revenue (product_category):")
print(revenue_by_product_category)

import matplotlib.pyplot as plt
import seaborn as sns

top_10_products = revenue_by_product_detail.head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='revenue', y='product_detail', data=top_10_products)
plt.title('Top 10 Products by Revenue (Product Detail)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Detail')
plt.tight_layout()
plt.show()

top_10_product_types = revenue_by_product_type.head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='revenue', y='product_type', data=top_10_product_types)
plt.title('Top 10 Products by Revenue (Product Type)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Type')
plt.tight_layout()
plt.show()

top_10_product_categories = revenue_by_product_category.head(10)

plt.figure(figsize=(10, 6))
```

```
sns.barplot(x='revenue', y='product_category', data=top_10_product_categories)
plt.title('Top 10 Products by Revenue (Product Category)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Category')
plt.tight_layout()
plt.show()

bottom_10_products = revenue_by_product_detail_b.head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='revenue', y='product_detail', data=bottom_10_products)
plt.title('Bottom 10 Products by Revenue (Product Detail)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Detail')
plt.tight_layout()
plt.show()

bottom_10_product_types = revenue_by_product_type_b.head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='revenue', y='product_type', data=bottom_10_product_types)
plt.title('Bottom 10 Products by Revenue (Product Type)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Type')
plt.tight_layout()
plt.show()

bottom_10_product_categories = revenue_by_product_category_b.head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='revenue', y='product_category', data=bottom_10_product_categories)
plt.title('Bottom 10 Products by Revenue (Product Category)')
plt.xlabel('Total Revenue')
plt.ylabel('Product Category')
plt.tight_layout()
plt.show()

efficiency = data.groupby('product_detail').agg({
    'revenue': 'sum',
    'transaction_qty': 'sum'
}).reset_index()

efficiency['revenue_per_unit'] = (
    efficiency['revenue'] / efficiency['transaction_qty'])

pareto = revenue_by_product_detail.copy()
pareto['cumulative_revenue'] = pareto['revenue'].cumsum()
pareto['cumulative_percentage'] = (
    pareto['cumulative_revenue'] / pareto['revenue'].sum() * 100
)

data.to_csv('cleaned_afficionado_dataset.csv', index=False)
```